

Documentación de Seguridad e Infraestructura - Proyecto SIRA

Este documento detalla la arquitectura de red, la gestión de usuarios y las medidas de protección implementadas en el proyecto SIRA (Sistema Integral de Riego Automático). Como parte del Trabajo de Fin de Grado (TFG) de ASIR, se ha diseñado esta infraestructura buscando un equilibrio entre seguridad, rendimiento y facilidad de despliegue.

1. Arquitectura de Red y Perímetro

Uso de Nginx como Proxy Inverso y API Gateway

Para el proyecto se ha configurado **Nginx** como el único punto de acceso externo al sistema, ejerciendo funciones de enrutador inteligente y cortafuegos de aplicación inicial.

- **Aislamiento de servicios:** Tanto el backend (FastAPI) como la base de datos (PostgreSQL) corren en una red privada de Docker (`sira-network`). Solo se puede acceder a ellos a través de Nginx, lo que evita ataques directos a los puertos 8000 o 5432 desde el host.
- **Ocultación de topología (Routing):** Nginx unifica todos los servicios bajo el mismo puerto. Enruta dinámicamente las peticiones sensibles (`/api`, `/docs`, `/redoc`) hacia FastAPI y delega el tráfico general al frontend, evitando que el usuario exterior conozca la distribución de puertos o servicios de la red interna.
- **Prevención de Denegación de Servicio (DoS) por saturación:** Se ha implementado un control estricto de subidas mediante la directiva `client_max_body_size 50M`, previniendo que un atacante envíe *payloads* gigantescos que colapsen la memoria del servidor o de la API.
- **Conexiones IoT seguras:** Se ha habilitado explícitamente el soporte de HTTP/1.1 y *Upgrade* de WebSockets, garantizando que el flujo de datos en tiempo real de los sensores no sufra cortes por *timeouts* o cierres abruptos.
- **Abstracción del puerto host:** El archivo `docker-compose.yml` abstrae la exposición exterior, permitiendo usar el puerto 8085 en local y mapearlo al puerto HTTP estándar 80 en AWS EC2, lo que facilita enormemente la configuración de los *Security Groups* de Amazon sin modificar el servidor web interno.
- **Trazabilidad y Auditoría de IP Real:** Nginx inyecta las cabeceras `X-Real-IP` y `X-Forwarded-For` hacia el frontend (PHP) y backend (FastAPI) en cada petición proxy. Esto permite mantener un registro de auditoría real (logs) con las direcciones IP exteriores reales de los clientes, impidiendo que el tráfico aparezca anonimizado bajo la IP interna del proxy.

Contenedores y Docker (Resiliencia e Infraestructura Segura)

- **Gestión de volúmenes persistentes:** Se han separado los datos de la base de datos y los logs de seguridad en volúmenes persistentes de Docker (`postgres_data`, `sira_security_history`), blindando el histórico de accesos y la base de datos frente a la destrucción física de los contenedores efímeros.
- **Hardening del Contenedor Nginx (Montaje de Solo Lectura):** El archivo de configuración de Nginx (`nginx.conf`) se monta en el contenedor en modo **Solo Lectura (:ro - Read-Only)**. Incluso en el escenario extremo de que el servidor web se viera comprometido, es físicamente imposible para un atacante alterar las directivas de red o inyectar redirecciones maliciosas en caliente.
- **Sincronización Crítica de Arranque (Healthchecks):** Para evitar fugas de sockets y caídas de servicio, la API de FastAPI no arranca hasta que la base de datos PostgreSQL supera con éxito el test de salud interna (`pg_isready -U postgres`), garantizando un inicio coordinado y robusto del ecosistema.
- **Resiliencia Automática (Alta Disponibilidad):** Todos los servicios de la pila Docker implementan políticas de autorreparación `restart: unless-stopped`. Ante cualquier caída abrupta del servicio debido a errores de ejecución o desbordamiento de memoria, Docker levanta el contenedor de forma automática en segundos.
- **Seguridad en el repositorio:** El archivo `.env` y las carpetas con datos sensibles están incluidos en el `.gitignore` para no subir secretos a GitHub.

2. Gestión de Usuarios y Accesos

Autenticación con JWT y Roles de Acceso

- **Sesión en servidor:** Aunque se usa JWT para la comunicación entre el frontend y la API, se guarda el token en la sesión de PHP por seguridad. Esto ayuda a prevenir ataques de tipo XSS que podrían robar el token si estuviera en el almacenamiento local del navegador.
- **Roles de usuario:** Se han implementado tres niveles de acceso: Root, Admin y Cliente, controlados mediante los "claims" del token JWT.
- **Principio de Privilegio Mínimo y Aislamiento del Superusuario (Root):** Para blindar el control de accesos, se ha implementado un estricto aislamiento de privilegios. Un usuario con rol de administración convencional (`admin`) tiene prohibido ver, listar o interactuar con la cuenta del superusuario (`root`) en el frontend de la plataforma. El perfil `root` queda invisible y fuera del alcance de los administradores convencionales, previniendo vectores de ataque de escalada de privilegios y manipulación de la cuenta maestra del sistema.

Protección de Contraseñas (Bcrypt)

- **Hashing:** Todas las contraseñas se guardan usando el algoritmo **Bcrypt con 12 rondas**, garantizando una robusta protección frente a ataques de fuerza bruta y precomputación de claves.
- **Sin texto plano:** En ninguna parte del sistema, incluyendo los scripts de inicialización SQL, se guardan contraseñas legibles.

Política de Complejidad y Forzado de Cambio (Iron Fortress)

- **Validación Dual de Contraseña:** Para evitar inyecciones y saltos de validación en la API, la robustez de las contraseñas se comprueba de forma redundante: tanto en el frontend (JavaScript interactivo para feedback visual de UX) como en el backend (expresiones regulares en `auth.py`).
 - *Cientes:* Contraseña de mínimo 8 caracteres con combinación de mayúsculas, minúsculas, números y símbolos especiales.
 - *Administradores y Root:* Requisito incrementado a un mínimo de 10 caracteres con la misma complejidad.
 - **Forzado de Cambio Inicial:** El campo flag `debe_cambiar_pw` del modelo de datos de base de datos obliga de forma estricta a cualquier usuario recién registrado a cambiar la contraseña predeterminada asignada por el administrador en su primer inicio de sesión.
 - **Bloqueo Inmediato de Cuentas Desactivadas (Soft Delete):** Las cuentas inactivas (`activa = False`) tienen el login prohibido de forma absoluta. Adicionalmente, si un usuario que tiene una sesión activa es desactivado por un administrador, el middleware de autorización del backend (`get_current_user`) detectará el cambio de estado en la siguiente consulta HTTP, revocando su token en tiempo real devolviendo un código `HTTP 401` y expulsándolo de la plataforma en milisegundos.
-

3. Control de Sesiones y Auditoría

Historial y Rotación de Claves

Para aumentar la seguridad de las cuentas, se han añadido estas funcionalidades:

- **Aislamiento Físico de Historiales:** Se guarda un registro de auditoría cifrado con las últimas 5 contraseñas de cada usuario de forma segura en archivos JSON dentro de un directorio aislado (`/app/security_history/`) del contenedor, protegiendo estos datos fuera del esquema relacional de la base de datos principal PostgreSQL.
- **Caducidad:** Las contraseñas caducan a los 90 días, obligando al usuario a cambiarlas.

Control de Inactividad y Sesión Única

- **Evitar sesiones duplicadas (Sesión Única):** Cada vez que alguien entra, se genera un ID de sesión único (UUIDv4) en base de datos. Si se entra desde otro dispositivo con la misma cuenta, la discrepancia de IDs invalida el token de la sesión anterior de inmediato (Iron Fortress).
 - **Tiempo de inactividad (Sliding Window):** Se ha configurado un sistema que cierra la sesión si no hay interacción del usuario en 30 minutos. Esto protege las cuentas ante descuidos físicos.
 - **Reseteo Seguro del Contador:** Para prevenir bloqueos recurrentes por inactividad residual de sesiones anteriores expiradas, al realizar un inicio de sesión exitoso el sistema actualiza de forma explícita el timestamp de `ultima_actividad` (`func.now()`), restableciendo el contador a cero y abriendo una ventana limpia de interacción.
 - **Cierre de sesión:** El botón de "Cerrar sesión" borra de forma activa el identificador en la base de datos de manera inmediata.
-

4. Seguridad en la Interfaz y Cabeceras HTTP

Validación y Filtrado

- **Mínimo JavaScript:** Se ha intentado usar poco JS para evitar vulnerabilidades. Solo se usa para dar feedback visual al usuario cuando crea una contraseña (comprobar que cumple los requisitos).
- **Escapado de datos:** En PHP se usa siempre `htmlspecialchars()` para evitar ataques de inyección de scripts (XSS) al mostrar datos.
- **Control Antifugas en Sesión PHP (Robustez de Flujo):** La lectura segura del token JWT mediante el operador de fusión nula (`$_SESSION['jwt_token'] ?? null;`) garantiza que PHP no emita advertencias internas ("Undefined array key") en pantalla ante sesiones expiradas. Esto neutraliza la fuga involuntaria de nombres del sistema de archivos del servidor ante el cliente final y asegura que las redirecciones de seguridad HTTP jamás queden bloqueadas por salida previa en cabeceras ("headers already sent").

Cabeceras Defensivas contra XSS y Secuestro de Tipos (MIME Sniffing)

- **Cabecera X-Content-Type-Options: nosniff:** Implementada a nivel de servidor web y proxy inverso para proteger las peticiones de los usuarios. Esta cabecera ordena al navegador que respete de manera estricta el tipo de archivo (`Content-Type`) declarado por el servidor, bloqueando de raíz que el navegador "olfatee" (MIME sniffing) y ejecute código JavaScript ejecutable oculto en archivos inofensivos (como imágenes o texto plano). Esto neutraliza vectores de ataque comunes de tipo Cross-Site Scripting (XSS) indirecto.

5. Seguridad en el Sistema IoT

Entorno Simulado y Telemetría Abierta

En la versión actual del proyecto (orientada a la demostración del TFG), la capa de telemetría IoT opera en un entorno simulado. Para facilitar el uso de la herramienta de "Simulación Climática" desde el dashboard y agilizar las pruebas de carga, los endpoints de ingesta de datos (`/api/v1/iot/mediciones/`) se mantienen abiertos dentro de la red interna sin requerir tokens de autenticación por dispositivo. La implementación de API Keys individuales por sensor queda como una ampliación planificada para futuras versiones del sistema en producción física.

Matriz de Riesgos

Riesgo	Impacto	Medida de Mitigación
Inyección SQL	Crítico	Uso de SQLAlchemy (ORM) que parametriza las consultas.
Fuerza Bruta	Alto	Contraseñas complejas y hashing con Bcrypt.
Fuga de datos	Crítico	Uso de <code>.gitignore</code> para archivos de configuración.
Robo de sesión	Alto	Cierre por inactividad y almacenamiento en sesión PHP.
Pérdida de datos	Crítico	Backups graduales (Anual/Mensual/Diario) con rotación automática.

6. Resiliencia y Recuperación

Para asegurar la continuidad del sistema, se ha implementado un script de **backup inteligente** (`backup_sira.sh`) automatizado mediante **Crontab**. Este sistema gestiona la rotación de copias de seguridad de forma automática, garantizando que siempre existan puntos de restauración históricos.

Configuración de automatización:

```
00 03 * * * /bin/bash /home/ubuntu/Proyecto_SIRA/scripts/backup_sira.sh >>
/home/ubuntu/sira_backups/log_cron.txt 2>&1
```

[!NOTE] Este documento resume el estado final de la seguridad del proyecto para la defensa del TFG.

Proyecto SIRA - Documentación Técnica

Última actualización: 12 de Mayo de 2026 (Versión 1.1)